

WEEK-2

Symmetric Encryption (using AES with OpenSSL)

Symmetric encryption uses the same key for encryption and decryption.

Step 1: Create a sample text file

```
echo "This is a secret message." > message.txt
```

```
$ pwsh
PowerShell 7.5.1

(mamatha123@kali)-[/home/mamatha123]
PS> echo "This is a secret message:" >message.txt
```

Step 2: Encrypt the file using AES-256-CBC

```
openssl enc -aes-256-cbc -salt -in message.txt -out encrypted_message.enc
```

You'll be prompted to enter a password. This password is the symmetric key.

```
(mamatha123@kali)-[/home/mamatha123]
PS> openssl enc -aes-256-cbc -salt -in message.txt -out encrypted_message.e
nc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Step 3: Decrypt the file

```
openssl enc -d -aes-256-cbc -salt -in encrypted_message.enc -out decrypted_message.txt
```

Use the same password used during encryption.

```
(mamatha123@kali)-[/home/mamatha123]
PS> openssl enc -d -aes-256-cbc -salt -in encrypted_message.enc -out decryp
ted_message.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Step 4: Verify

```
cat decrypted_message.txt
```

```
(mamatha123@kali)-[/home/mamatha123]  
PS> cat decrypted_message.txt  
This is a secret message:
```

Output:

```
(mamatha123@kali)-[/home/mamatha123]
PS> echo "This is a secret message:" >message.txt

(mamatha123@kali)-[/home/mamatha123]
PS> openssl enc -aes-256-cbc -salt -in message.txt -out encrypted_message.e
nc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(mamatha123@kali)-[/home/mamatha123]
PS> openssl enc -d -aes-256-cbc -salt -in encrypted_message.enc -out decryp
ted_message.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(mamatha123@kali)-[/home/mamatha123]
PS> cat decrypted_message.txt
This is a secret message:
```

Asymmetric Encryption using RSA

Asymmetric encryption uses a public/private key pair. RSA is a common algorithm.

Step 1: Generate RSA key pair

Generate private key

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

```
(masat@kali:~)$ openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

Extract public key

openssl rsa -pubout -in private_key.pem -out public_key.pem

```
(mamatha123@kali)~$ openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key
```

Step 2: Encrypt a file with the public key

echo "Confidential message for asymmetric encryption." > rsa_message.txt

```
(mamatha123@kali)~$ echo "Confidential message for asymmetric encryption." > rsa_message.txt
```

Encrypt with public key

openssl pkeyutl -encrypt -inkey public_key.pem -pubin -in rsa_message.txt -out rsa_encrypted.bin

```
(mamatha123@kali)~$ openssl pkeyutl -encrypt -inkey public_key.pem -pubin -in rsa_message.txt -out rsa_encrypted.bin
```

Step 3: Decrypt the file with the private key

openssl pkeyutl -decrypt -inkey private_key.pem -in rsa_encrypted.bin -out rsa_decrypted.txt

```
(mamatha123@kali)~$ openssl pkeyutl -decrypt -inkey private_key.pem -in rsa_encrypted.bin -out rsa_decrypted.txt
```

Step 4: Verify

cat rsa_decrypted.txt

```
(mamatha123@kali)~$ cat rsa_decrypted.txt
Confidential message for asymmetric encryption.
```

Output:

Confidential message for asymmetric encryption.

[illegible]

Symmetric: AES-GCM with Python (PyCryptodome)

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
def aes_gcm_encrypt(plaintext: bytes) -> dict:
    key = get_random_bytes(32) # 256-bit key
    nonce = get_random_bytes(12) # 96-bit nonce recommended for GCM
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    ciphertext, tag = cipher.encrypt_and_digest(plaintext)
    return {"key": key, "nonce": nonce, "ciphertext": ciphertext, "tag": tag}
def aes_gcm_decrypt(key: bytes, nonce: bytes, ciphertext: bytes, tag: bytes) -> bytes:
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    return cipher.decrypt_and_verify(ciphertext, tag)
if __name__ == "__main__":
    msg = b"Confidential message \xF0\x9F\x94\x90"
    bundle = aes_gcm_encrypt(msg)
    recovered = aes_gcm_decrypt(bundle["key"], bundle["nonce"], bundle["ciphertext"], bundle["tag"])
    assert recovered == msg
    print("AES-GCM OK")
```

Output: AES-GCM OK

Asymmetric: RSA-OAEP with Python

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
from Crypto.Hash import SHA256
def gen_rsa_keypair(bits=2048):
    key = RSA.generate(bits)
    priv_pem = key.export_key()
    pub_pem = key.publickey().export_key()
    return priv_pem, pub_pem
def rsa_encrypt(pub_pem: bytes, data: bytes) -> bytes:
    pub_key = RSA.import_key(pub_pem)
    cipher = PKCS1_OAEP.new(pub_key, hashAlgo=SHA256)
    return cipher.encrypt(data)
def rsa_decrypt(priv_pem: bytes, ct: bytes) -> bytes:
    priv_key = RSA.import_key(priv_pem)
    cipher = PKCS1_OAEP.new(priv_key, hashAlgo=SHA256)
    return cipher.decrypt(ct)
if __name__ == "__main__":
    priv, pub = gen_rsa_keypair()
    msg = b"Small secret"
    ct = rsa_encrypt(pub, msg)
    pt = rsa_decrypt(priv, ct)
    assert pt == msg
    print("RSA-OAEP OK")
```

Output: RSA-OAEP OK

